

MEMORIA DE ANALISIS Y PREGUNTAS DE REFLEXION

SERGIO BARROSO MILAN

28/05/2026

INDICE

<u>Introducción</u>	2
<u>Identificación de Entidades y Atributos</u>	3
<u>Preguntas de reflexión</u>	11

Introducción

El proyecto NexShop Group S.A. consiste en el análisis, diseño e implementación desde cero de una base de datos relacional sólida y eficiente para una empresa de distribución y venta al por menor fundada en 2015 y con sede en Valencia. La compañía cuenta con una estructura de negocio híbrida que integra una plataforma de comercio electrónico activa (nexshop.es) junto con una red de tres tiendas físicas distribuidas estratégicamente en Valencia, Madrid y Barcelona. A pesar de que ambas líneas comerciales comparten un catálogo unificado de más de 2.000 productos organizados jerárquicamente en categorías y subcategorías, sus operaciones diarias, equipos humanos y gestión de inventarios funcionan de manera independiente.

El objetivo principal es unificar y dar soporte digital a los requerimientos de los departamentos de Operaciones, Logística, Atención al Cliente y Marketing. El sistema propuesto no solo debe registrar con precisión los procesos de venta presencial, pedidos en línea y control de existencias por ubicación, sino que también está diseñado para gestionar la trazabilidad histórica de los precios de coste de proveedores, las campañas de promociones temporales, los tickets de incidencias y un programa avanzado de fidelización de clientes basado en la acumulación y canje de puntos. Todo ello se estructurará bajo un **modelo relacional** normalizado que garantice la integridad de los datos, optimice el rendimiento de las consultas y resuelva eficazmente las problemáticas operativas actuales de la organización

1. Tu mision

No hay tabla de pistas. No hay lista de entidades.

Tu unica fuente de informacion es el texto que encontraras a continuacion.

Lee con atencion, toma notas y construye el modelo desde cero.

WE MAKE IT POSSIBLE

Eres analista de bases de datos en una consultora tecnologica. Tu primer encargo real es disenar y construir la base de datos de NexShop Group S.A., una empresa con varios anos de historia y muchas capas de negocio. El cliente no sabe nada de bases de datos: te describira su empresa como lo haria cualquier director de operaciones, con lenguaje de negocio, no tecnico. Tu trabajo es escuchar, interpretar y traducirlo a un modelo relacional solido.

Se te proporcionan tres fuentes de informacion: una descripcion general de la empresa, el acta de una reunion con los responsables de cada departamento, y un fragmento de correos internos reales. De todo ello deberas extraer las entidades, atributos y relaciones necesarias.

Identificación de Entidades y Atributos

Se han identificado las siguientes entidades y atributos:

Sedes. la empresa tiene una sede central en Valencia (logística y compras) y una red de tres tiendas físicas (Valencia, Madrid y Barcelona).

Atributos:

- **id_sede** (INT, PK, Autoincremental): **[Lo propongo yo]** Clave primaria necesaria para identificar inequívocamente cada sede.
- **nombre** (VARCHAR): **[Lo pide el cliente]** Para diferenciar si es "Sede Central", "Tienda Madrid", etc.
- **ciudad** (VARCHAR): **[Lo pide el cliente]** Valencia, Madrid, Barcelona.
- **tipo** (ENUM('Tienda', 'Almacen')): **[Lo propongo yo]** Para distinguir las reglas de negocio de una tienda física frente al almacén central.

Empleados. Se menciona que hay 47 empleados repartidos en las sedes y que realizan roles específicos (encargados, vendedores, responsables de almacén, agentes de atención al cliente).

Atributos:

- **id_empleado** (INT, PK, Autoincremental): **[Lo propongo yo]** Identificador único.
- **dni** (VARCHAR, UNIQUE): **[Lo pide el cliente]** Documento de identidad.
- **nombre** (VARCHAR): **[Lo pide el cliente]** Nombre completo.
- **email_corporativo** (VARCHAR, UNIQUE): **[Lo pide el cliente]** Correo de la empresa.
- **fecha_incorporacion** (DATE): **[Lo pide el cliente]** Registro de cuándo entró a trabajar.
- **id_sede** (INT, FK): **[Lo pide el cliente]** Relación para saber a qué sede están asignados.
- **rol_cargo** (ENUM('Encargado', 'Vendedor', 'Responsable Almacen', 'Agente Atencion', 'Logistica', 'Compras')) — **[Lo propongo yo]:** *Lo añadimos como un tipo ENUM para restringir los puestos exactamente a los que pide el texto, evitando que se introduzcan roles inventados y permitiendo controlar la lógica del negocio sin crear tablas auxiliares.*

Categorías. El catálogo de productos está organizado en categorías y subcategorías.

Atributos:

- **id_categoria** (INT, PK, Autoincremental): **[Lo propongo yo]** Identificador de la categoría principal.
- **nombre** (VARCHAR): **[Lo pide el cliente]** Ejemplo: "Informática".

Subcategorías. Un producto pertenece a una subcategoría, y esta a su vez pertenece a una categoría (jerarquía). El cliente especifica que un producto nunca puede pertenecer a dos subcategorías distintas.

Atributos:

- **id_subcategoria** (INT, PK, Autoincremental): **[Lo propongo yo]** Identificador único.
- **nombre** (VARCHAR): **[Lo pide el cliente]** Ejemplo: "Portátiles".
- **id_categoria** (INT, FK): **[Lo propongo yo]** Para enlazar la subcategoría con su categoría padre (Informática -> Portátiles).

Productos. El catálogo de NexShop.

Atributos:

- **id_producto** (INT, PK, Autoincremental): **[Lo propongo yo]** Identificador del producto.
- **referencia** (VARCHAR, UNIQUE): **[Lo pide el cliente]** El código único de producto que menciona el texto.
- **nombre** (VARCHAR): **[Lo propongo yo]** Necesario para saber qué producto es (ej. "Portátil Gaming X").
- **pvp_actual** (DECIMAL): **[Lo pide el cliente]** El precio de venta al público base.
- **id_subcategoria** (INT, FK): **[Lo pide el cliente]** Clave foránea para asegurar que un producto pertenece a una única subcategoría.

Histórico precios. El cliente indica explícitamente que el PVP puede variar con el tiempo y el sistema debe ser capaz de mostrar el historial completo de precios.

Atributos:

- **id_historico_precio** (INT, PK, Autoincremental): **[Lo propongo yo]** Clave primaria.

- **id_producto** (INT, FK): **[Lo pide el cliente]** El producto al que cambia el precio.
- **precio** (DECIMAL): **[Lo pide el cliente]** El PVP que tenía en ese rango de fechas.
- **fecha_inicio** (DATE) y **fecha_fin** (DATE, NULL): **[Lo propongo yo]** Para acotar temporalmente la validez de ese precio. Si **fecha_fin** es NULL, es el precio vigente.

Promociones. Marketing lanza promociones periódicas con descuentos porcentuales sobre productos en rangos de fechas concretos.

Atributos:

- **id_promocion** (INT, PK, Autoincremental): **[Lo propongo yo]** Identificador de la promoción.
- **nombre** (VARCHAR): **[Lo propongo yo]** Para identificar la campaña (ej. "Black Friday").
- **descuento_porcentaje** (INT o DECIMAL): **[Lo pide el cliente]** El porcentaje a aplicar.
- **fecha_inicio** (DATE) y **fecha_fin** (DATE): **[Lo pide el cliente]** Rango de vigencia de la promoción.

Promociones_Productos: Un producto puede tener varias promociones al año y una promoción se aplica a determinados productos. Guardará **id_promocion** y **id_producto**. (tabla intermedia).

Proveedores. Suministran los productos al almacén central y tienen asignado un representante comercial de NexShop.

Atributos:

- **id_proveedor** (INT, PK, Autoincremental): **[Lo propongo yo]** Clave primaria.
- **nombre** (VARCHAR): **[Lo propongo yo]** Nombre de la empresa proveedora.
- **id_empleado_representante** (INT, FK): **[Lo pide el cliente]** Enlace al empleado de NexShop que gestiona la relación comercial.

Condiciones_proveedor. Un producto puede ser suministrado por varios proveedores. Para cada combinación producto-proveedor se negocia un precio de coste y un plazo de entrega. Además, cambian periódicamente y se debe guardar el histórico.

Atributos:

- **id_condicion** (INT, PK, Autoincremental): **[Lo propongo yo]** Clave única para el histórico.
- **id_producto** (INT, FK) e **id_proveedor** (INT, FK): **[Lo pide el cliente]** La combinación de ambos.
- **precio_coste** (DECIMAL): **[Lo pide el cliente]** Precio pactado.
- **plazo_entrega_dias** (INT): **[Lo pide el cliente]** Días estimados de entrega.
- **fecha_negociacion** (DATE): **[Lo propongo yo]** Fecha en la que entraron en vigor estas condiciones para poder trazar el histórico.

Stock_ubicación. El stock de cada producto se controla de forma independiente por cada tienda y por el almacén central.

Atributos:

- **id_producto** (INT, PK, FK): **[Lo pide el cliente]** Producto en cuestión.
- **id_sede** (INT, PK, FK): **[Lo pide el cliente]** Ubicación física.
- **cantidad** (INT): **[Lo pide el cliente]** Nivel de stock actual en esa ubicación concreta.

Clientes. Son los usuarios del sistema. Los hay registrados (online) y no registrados (compras anónimas en tienda física).

Atributos:

- **id_cliente** (INT, PK, Autoincremental): **[Lo propongo yo]** Clave primaria que usaremos tanto para los registrados como para los anónimos (ventas mostrador).
- **nombre** (VARCHAR, NULL): **[Lo pide el cliente]** Puede ser nulo si compra en tienda sin registrarse.
- **apellidos** (VARCHAR, NULL): **[Lo pide el cliente]**
- **email** (VARCHAR, UNIQUE, NULL): **[Lo pide el cliente]** Clave de acceso online.
- **contrasena** (VARCHAR, NULL): **[Lo pide el cliente]**
- **fecha_nacimiento** (DATE, NULL): **[Lo pide el cliente]** Opcional para marketing.
- **es_registrado** (BOOLEAN): **[Lo propongo yo]** Flag para saber rápidamente si es una cuenta activa online o un perfil temporal de tienda.

- **saldo_puntos_actual** (INT, DEFAULT 0) — **[Lo propongo yo]**: *Lo añadimos explícitamente porque, aunque es un dato calculado, lo necesitamos físicamente en la tabla para que las consultas de saldo en la web o en la caja de la tienda no saturen el servidor calculando históricos en tiempo real.*

Direcciones_clientes. Un cliente registrado puede tener múltiples direcciones (domicilio, trabajo, etc.).

Atributos:

- **id_direccion** (INT, PK, Autoincremental): **[Lo propongo yo]** Clave primaria.
- **id_cliente** (INT, FK): **[Lo pide el cliente]** Cliente al que pertenece la dirección.
- **tipo_direccion** (VARCHAR): **[Lo pide el cliente]** "Domicilio", "Trabajo", etc.
- **calle** (VARCHAR), **numero** (VARCHAR), **piso** (VARCHAR, NULL), **codigo_postal** (VARCHAR), **ciudad** (VARCHAR), **pais** (VARCHAR): **[Lo pide el cliente]** Datos de ubicación exactos.

Pedidos online. Pedidos realizados a través de nexshop.es. Requieren asociarse a un cliente registrado y a una dirección de envío específica.

Atributos:

- **id_pedido** (INT, PK, Autoincremental): **[Lo propongo yo]** Número de pedido.
- **id_cliente** (INT, FK): **[Lo pide el cliente]** Cliente que compra.
- **id_direccion** (INT, FK): **[Lo pide el cliente]** Dirección elegida para la entrega.
- **fecha_pedido** (DATETIME): **[Lo propongo yo]** Momento exacto de la compra.
- **estado** (VARCHAR): **[Lo propongo yo]** Ej: 'Pendiente', 'Procesado', 'Dividido'.

Líneas_pedido_online. Pedidos realizados a través de nexshop.es. Requieren asociarse a un cliente registrado y a una dirección de envío específica.

Atributos:

- **id_pedido** (INT, PK, Autoincremental): **[Lo propongo yo]** Número de pedido.

- **id_cliente** (INT, FK): **[Lo pide el cliente]** Cliente que compra.
- **id_direccion** (INT, FK): **[Lo pide el cliente]** Dirección elegida para la entrega.
- **fecha_pedido** (DATETIME): **[Lo propongo yo]** Momento exacto de la compra.
- **estado** (VARCHAR): **[Lo propongo yo]** Ej: 'Pendiente', 'Procesado', 'Dividido'.

Envíos. Logística indica que un pedido online puede dividirse en varios envíos parciales si el stock sale de almacenes diferentes. Cada envío tiene sus propios datos logísticos.

Atributos:

- **id_envio** (INT, PK, Autoincremental): **[Lo propongo yo]** Identificador del envío.
- **id_pedido** (INT, FK): **[Lo pide el cliente]** Pedido de origen.
- **id_sede_origen** (INT, FK): **[Lo pide el cliente]** Almacén o tienda desde donde sale el stock.
- **numero_seguimiento** (VARCHAR): **[Lo pide el cliente]** Código del transportista.
- **transportista** (VARCHAR): **[Lo pide el cliente]** Empresa de transporte.
- **fecha_estimada_entrega** (DATE): **[Lo pide el cliente]**

Líneas Envío: **[Lo propongo yo]** Al poder dividirse un pedido, necesitamos saber qué productos de las líneas de pedido van en este envío concreto y en qué cantidad. (tabla intermedia).

Ventas presenciales. (Tickets de ventas) En las tiendas físicas se generan tickets de venta directos, no pedidos online. Requieren saber qué empleado hizo la venta y desde qué sede física. El cliente puede ser anónimo o registrado.

Atributos:

- **id_ticket** (INT, PK, Autoincremental): **[Lo propongo yo]** Número de ticket de caja.
- **id_sede** (INT, FK): **[Lo pide el cliente]** Tienda física donde se vende.
- **id_empleado** (INT, FK): **[Lo pide el cliente]** Vendedor que atiende.
- **id_cliente** (INT, FK, NULL): **[Lo pide el cliente]** Será NULL si el cliente no está registrado, o tendrá el ID si se vincula.

- **fecha_venta** (DATETIME): **[Lo propongo yo]** Fecha y hora del cobro.

Líneas_ticket_presencial. Desglose de los productos vendidos en la tienda física.

Atributos:

- **id_linea_ticket** (INT, PK): **[Lo propongo yo]**
- **id_ticket** (INT, FK): **[Lo propongo yo]**
- **id_producto** (INT, FK): **[Lo pide el cliente]**
- **cantidad** (INT): **[Lo pide el cliente]** Cantidad comprada.
- **precio_unitario_aplicado** (DECIMAL): **[Lo propongo yo]** Precio final cobrado en caja.

Devoluciones presenciales. Las devoluciones en tiendas físicas generan un documento de devolución vinculado al ticket original.

Atributos:

- **id_devolucion** (INT, PK, Autoincremental): **[Lo propongo yo]**
- **id_ticket** (INT, FK): **[Lo pide el cliente]** Ticket de origen.
- **fecha_devolucion** (DATETIME): **[Lo propongo yo]**
- **motivo** (VARCHAR): **[Lo propongo yo]** Información útil para el negocio.

(Las líneas de devolución detallarán los productos y cantidades devueltas).

Transferencias_stock. Movimientos internos entre tiendas o desde el almacén central para reponer stock de productos.

Atributos:

- **id_transferencia** (INT, PK, Autoincremental): **[Lo propongo yo]**
- **fecha** (DATE): **[Lo pide el cliente]**
- **id_sede_origen** (INT, FK): **[Lo pide el cliente]** Sede emisora.
- **id_sede_destino** (INT, FK): **[Lo pide el cliente]** Sede receptora.
- **id_producto** (INT, FK): **[Lo pide el cliente]** Qué se transfiere.
- **cantidad** (INT): **[Lo pide el cliente]** Cuántas unidades.
- **id_empleado_autoriza** (INT, FK): **[Lo pide el cliente]** Responsable que firma la transferencia.

Tickets incidencia. Abiertos por Atención al Cliente para quejas, consultas o devoluciones online (que generan recogida).

Atributos:

- **id_ticket_incidencia** (INT, PK, Autoincremental): **[Lo propongo yo]**
- **asunto** (VARCHAR) y **descripcion** (TEXT): **[Lo pide el cliente]** Detalles del problema.
- **fecha_apertura** (DATETIME): **[Lo pide el cliente]**
- **estado** (ENUM('abierto', 'en gestion', 'resuelto')): **[Lo pide el cliente]** Estado actual.
- **id_empleado_agente** (INT, FK): **[Lo pide el cliente]** Agente que lo tramita.
- **id_pedido** (INT, FK, NULL): **[Lo pide el cliente]** Opcional, ya que a veces son consultas generales sin pedido asociado.
- **fecha_cierre** (DATETIME, NULL): **[Lo pide el cliente]** Nulo hasta que se resuelva.
- **nota_resolucion** (TEXT, NULL): **[Lo pide el cliente]** Explicación de cómo se solucionó.

Valoraciones. Sistema de puntuación y comentarios de clientes registrados sobre productos comprados.

Atributos:

- **id_valoracion** (INT, PK, Autoincremental): **[Lo propongo yo]**
- **id_cliente** (INT, FK): **[Lo pide el cliente]** Cliente que opina.
- **id_producto** (INT, FK): **[Lo pide el cliente]** Producto valorado.
- **puntuacion** (INT): **[Lo pide el cliente]** Restricción CHECK de 1 a 5.
- **comentario** (TEXT, NULL): **[Lo pide el cliente]** Opinión escrita.
- **es_verificada** (BOOLEAN): **[Lo pide el cliente]** Para filtrar si hay compra confirmada o si proviene del histórico antiguo sin compra previa. (El cliente indica que solo se puede valorar una vez por producto, por lo que aplicaremos una clave única compuesta por **id_cliente** + **id_producto**).

Movimientos puntos. Historial detallado del programa de fidelización de puntos para poder calcular el saldo actual de cada cliente.

Atributos:

- **id_movimiento** (INT, PK, Autoincremental): **[Lo propongo yo]**
- **id_cliente** (INT, FK): **[Lo pide el cliente]** Cliente afectado.
- **id_pedido** (INT, FK, NULL): **[Lo pide el cliente]** El pedido online donde se generaron o canjearon los puntos (puede ser NULL si Marketing decide regalar puntos directos por cumpleaños o eventos futuros).

- **cantidad_puntos** (INT): **[Lo pide el cliente]** Puntos del movimiento.
- **tipo_movimiento** (ENUM ('ganados', 'canjeados')): **[Lo pide el cliente]** Indica si suman o restan.
- **fecha_movimiento** (DATETIME): **[Lo pide el cliente]** Cuándo ocurrió.

Preguntas de Reflexión

1. **El texto menciona que un pedido online puede generar varios envíos. Explica como lo has modelado y por qué.**

Se ha resuelto mediante una relación de uno a muchos (1:N) entre la entidad “Pedidos_Online” y la entidad “Envíos”. Un pedido puede registrar múltiples envíos, pero cada envío pertenece a un único pedido de origen.

Se añade una tabla intermedia llamada `Lineas_Envio` entre “Envíos” y “Lineas_Pedido_Online”.

El departamento de logística indica que, si una sola sede no dispone de stock suficiente, el pedido se divide para enviar los artículos desde diferentes almacenes. Al meter la tabla `Lineas_Envio`, podemos registrar exactamente “**qué cantidad de qué producto**” sale de qué almacén, asociándole a cada paquete su propio número de seguimiento, transportista y fecha estimada de entrega de forma independiente. Si metiéramos los datos de envío directamente en la tabla del pedido, sería imposible gestionar las divisiones de stock.

2. **Las ventas presenciales y los pedidos online son procesos distintos, pero comparten productos y clientes. ¿Has optado por una sola tabla o dos tablas separadas? Justifica tu decisión con ventajas e inconvenientes de cada opción.**

He optado por dos tablas separadas de “Pedidos_Online” y “Ventas_Presenciales” (Tickets), pero vinculadas a entidades comunes para los “Clientes” y los “Productos”.

Los pedidos online requieren obligatoriamente una dirección de envío (`id_direccion`), un estado del pedido (Pendiente, enviado) y generan envíos logísticos. Las ventas en tienda física ocurren en el acto, requieren registrar el empleado (vendedor) y la sede física concreta donde se realiza la venta, y no pasan por estados de envío.

Separarlas evita tener una cantidad masiva de valores nulos. Además, Las consultas de caja de las tiendas físicas no interfieren con el flujo de la web.

Si metemos todo junto, los registros de tienda física tendrían vacíos los campos de dirección de envío, transportista y número de seguimiento. A su vez, los pedidos online no tendrían inicialmente un empleado asignado en la caja.

Si Marketing quiere sacar un informe globalizado de ingresos totales de la empresa, tendrá que unificar ambas tablas mediante una consulta con UNION.

3. El histórico de precios de un producto y el histórico de promociones son dos cosas distintas. Como las diferencias en tu modelo?

Se diferencian separando el valor comercial base del producto de las estrategias temporales de marketing:

- **Histórico de Precios (Historico_Precios):** Registra la evolución del **PVP base (Precio de Venta al Público) sin descuentos**. Si un ordenador portátil sube de 800€ a 850€ de forma permanente debido a la inflación o al coste de componentes, este cambio se registra aquí con su respectiva fecha de inicio y fin.
- **Histórico de Promociones (Promociones y Promociones_Productos):** Registra eventos temporales y específicos de marketing que aplican un descuento porcentual sobre el PVP vigente en ese momento. Por ejemplo, un "10% de descuento en portátiles por el Black Friday del 20 al 30 de noviembre".

Para saber cuánto pagó un cliente en una fecha exacta, el sistema busca el PVP base en "Historico_Precios" para esa fecha y, si existe una promoción activa en la tabla intermedia en ese mismo rango temporal, aplica el descuento. Este resultado final calculado se almacena de forma estática en el campo "precio_unitario_aplicado" de la línea de la venta para que quede blindado ante futuros cambios comerciales.

4. El saldo de puntos de fidelización 'debe poder calcularse siempre desde el histórico de movimientos'. ¿Qué implicación tiene esto en el diseño? Guardarías también un campo de saldo_actual en la tabla de clientes? ¿Por qué si o por qué no?

Exige la creación obligatoria de una tabla de auditoría (Movimientos_Puntos) donde cada inserción es inmutable. Cada registro guarda la fecha, los puntos sumados o restados y el tipo de operación. El diseño debe garantizar que la suma de los

movimientos de tipo 'ganados' menos los de tipo 'canjeados' equivalga exactamente al estado financiero de puntos del cliente.

¿Guardaría el campo `saldo_actual` en la tabla “Clientes”? Sí, lo guardaría (como un campo redundante calculado/calculable), pero con condiciones.

Por una cuestión de rendimiento técnico. Si la empresa tiene miles de clientes con un historial largo de compras, calcular el saldo haciendo un `SUM()` de todo el histórico de movimientos cada vez que un cliente inicia sesión en la web o va a pagar en caja generaría un impacto severo y lento en la base de datos.

Para evitar la pérdida de integridad (que el saldo no cuadre con el histórico), este campo de la tabla clientes **nunca se modifica a mano**; se actualiza de forma automatizada mediante un **Trigger (disparador)** en MySQL. Cada vez que se inserta un registro en `Movimientos_Puntos`, el Trigger recalcula y actualiza el campo `saldo_actual` en la ficha del cliente, asegurando rendimiento rápido y consistencia matemática absoluta.

5. Un cliente sin cuenta (compra presencial) puede luego vincularse a una cuenta online. Como has previsto esto en tu modelo?

Cuando un cliente compra en tienda física de forma anónima, el sistema asigna la venta a un registro único genérico en la tabla clientes (por ejemplo, `id_cliente = 1`, llamado "Cliente de Mostrador Anónimo").

Si el cliente no quiere ser anónimo, pero no tiene cuenta online, el vendedor puede crearle una ficha básica en la tabla `Clientes` registrando únicamente los datos que proporcione (o un identificador interno de su ticket). En ese estado, el campo “`es_registrado`” estará en `FALSE`, y los campos de “`email`” y “`contrasena`” estarán vacíos (`NULL`). El “`id_ticket`” de su venta se asocia a este nuevo ID de cliente personalizado.

Cuando el cliente decide registrarse en la plataforma online, el sistema buscará si existe una ficha previa (por ejemplo, validando un campo común como el DNI si se pidió en tienda, o mediante un código de vinculación impreso en su ticket físico). Al hacer la vinculación, se actualiza ese mismo registro: se rellenan los campos “`email`” y “`contrasena`”, y el flag “`es_registrado`” pasa a `TRUE`. De este modo, todas las ventas presenciales que ya estaban apuntadas a su “`id_cliente`” se quedan unificadas automáticamente con su nuevo perfil online sin romper ninguna clave foránea.

6. Hay algun requisito del texto que hayas decidido NO implementar o simplificar? Si es así, explica por qué y que impacto tiene.

Requisito simplificado: La gestión del rol de los empleados dentro de las sedes. El enunciado especifica detalladamente los puestos de trabajo en las sedes: *"Cada tienda física tiene un encargado, varios vendedores y un responsable de almacén"*. También menciona que la sede central coordina las compras y la logística. Sin embargo, para evitar una sobre complicación del modelo con tablas de roles, permisos o herencias extensibles, he decidido simplificarlo centralizando todo en la entidad "Empleados" con un campo atributo de texto o enumerado llamado "rol_cargo".

El impacto es mínimo, el sistema puede identificar perfectamente a través de restricciones y consultas quién autorizó una transferencia (comprobando si su rol es 'Responsable de Almacén') o quién realizó una venta (rol 'Vendedor') sin necesidad de crear complejas jerarquías de tablas. El diseño se mantiene limpio, eficiente y perfectamente funcional para cumplir con todas las fases del proyecto.